UNIVERSITÉ ALIOUNE DIOP DE BAMBEY
UFR Sciences & Technologies | Master 2 Big Data & Intelligence Artificielle
Année universitaire 2025-2026

PROJET TUTORÉ — CODES SOURCES COMPLETS

⚡ Watt-SN

Intelligence Énergétique & Prédiction de Délestage

Énergie & Transition Solaire

Durée	4 semaines
Équipe	2 étudiants
Domaine	Énergie & Transition Solaire
Standard	M2 Approche Industrie | New Deal Technologique Sénégal
Enseignant	Mr Ahmed Ben Sidy Bouya SEYE — Senior Big Data & AI Engineer | Groupe Sonatel

NiFi • Kafka • Spark Streaming • Pandera • HBase • Hive • Airflow • MLlib • Privacy by Design

1. Contexte et Objectifs

Le Sénégal produit ~800 MW pour une demande de ~1 200 MW en pointe, générant des délestages fréquents qui coûtent ~2% du PIB annuellement. Le plan Sénégal 2035 vise 40% d'énergies renouvelables. Ce projet construit un pipeline de prédiction de délestage et d'optimisation de l'intégration solaire dans le réseau national.

> ℹ️ **New Deal Technologique Sénégal**
> Ce projet s'inscrit dans la Stratégie Sénégal Numérique 2025-2035 et le Plan Sénégal Émergent. L'objectif est de former des ingénieurs Big Data capables de construire des solutions Data-Driven directement applicables aux défis de développement du Sénégal.

1.1 Objectifs Techniques

#	Composant	Description
1	**Ingestion multi-sources**	consommation Senelec, production solaire (GIS), météo ANACIM
2	**Prédiction de délestage 24h à l'avance (classification ROUGE/ORANGE/VERT)**	Prédiction de délestage 24h à l'avance (classification ROUGE/ORANGE/VERT)
3	**Optimisation de l'injection solaire par zone (Dakar, Thiès, Saint-Louis, Ziguinchor)**	Optimisation de l'injection solaire par zone (Dakar, Thiès, Saint-Louis, Ziguinchor)
4	**Privacy by Design**	anonymisation des compteurs industriels (contrats confidentiels)
5	**Dashboard Airflow**	alertes délestage + recommandations de délestage tournant équitable

1.2 Architecture End-to-End

📄 **Architecture**

```
[ SOURCES ]  ──▶  [ NiFi ]  ──▶  [ KAFKA ]  ──▶  [ SPARK ML ]  ──▶  [ STORAGE ]
 Senelec API      Orchestration   Topic:          Prédiction          HBase/HDFS
 Météo ANACIM     & Routage       watt_conso      Délestage            Gold Layer
 Capteurs GIS                     watt_solaire    Classification          │
 (panneaux)                                              │          [ ANALYTICS ]
                                                         ▼            Apache Hive
                                              [ ALERT LAYER ]       vue_risque_zone
                                              Délestage ROUGE       vue_solaire_opt
```

2. Prérequis & Infrastructure

> ℹ️ **Configuration minimale**
> RAM : 16 Go. CPU : 4 cœurs. OS : Ubuntu 22.04 LTS. Docker Engine 24.x. Java 11 requis par Spark.

📄 **setup.sh**

```
# Démarrage infrastructure
mkdir -p data dags models nifi_templates
docker compose up -d zookeeper && sleep 10
docker compose up -d kafka nifi hbase hive-metastore && sleep 30
docker compose up -d spark-master spark-worker airflow

# Initialiser HBase (une seule fois)
python hbase_setup.py

# Démarrer le simulateur
python kafka_producer_watt_sn.py &

# Interfaces web :
#   NiFi    : http://localhost:8081
#   Spark   : http://localhost:8080
#   HBase   : http://localhost:16010
#   Airflow : http://localhost:8082
```

📄 **requirements.txt**

```
# requirements.txt — Watt-SN
pyspark==3.3.2
kafka-python==2.0.2
pandera==0.17.2
pandas==2.0.3
apache-airflow==2.7.3
mlflow==2.8.1
pyhive==0.7.0
happybase==1.2.0
matplotlib==3.7.3
seaborn==0.12.2
faker==30.1.0
scipy==1.11.4
numpy==1.24.4
```

3. Codes Sources Complets & Fonctionnels

3.1 Initialisation HBase — hbase_setup.py

> ⚠️ **Fichier obligatoire**
> Crée les 3 tables HBase de Watt-SN. À exécuter avant le démarrage du pipeline.

📄 **hbase_setup.py**

```
# hbase_setup.py — Tables HBase Watt-SN
import happybase, logging
logger = logging.getLogger('HBaseSetup')

def create_watt_sn_tables():
    conn = happybase.Connection('hbase', port=9090, timeout=10000)
    conn.open()
    tables = {
        b'watt:conso_temps_reel': {'b'meta': {'max_versions': 1}},  # Consommation par
zone (TTL 48h)
        b'watt:alertes_delestage': {'b'meta': {'max_versions': 1}},  # Alertes
ROUGE/ORANGE (TTL 24h)
        b'watt:solaire_injection': {'b'meta': {'max_versions': 1}},  # Production solaire
+ forecast
    }
    existantes = [t.decode() for t in conn.tables()]
    for nom_b, fam in tables.items():
        nom = nom_b.decode()
        if nom not in existantes:
            conn.create_table(nom_b, fam)
            logger.info(f'Table {nom} créée ✓')
    conn.close()

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    create_watt_sn_tables()
```

3.2 Simulateur de Données — kafka_producer

> ℹ️ **Données simulées réalistes**
> Le simulateur génère des données calées sur les réalités de Énergie & Transition Solaire au Sénégal. Les champs PII (noms, IDs) sont présents dans le simulateur et seront anonymisés par la Privacy Layer Spark.

📄 **kafka_producer_watt_sn.py**

```
# kafka_producer_watt.py — Simulateur consommation Senelec + production solaire
from kafka import KafkaProducer
import json, random, time, uuid
from datetime import datetime
import numpy as np

random.seed(42); np.random.seed(42)
ZONES =
['DAKAR_NORD','DAKAR_SUD','THIES','SAINT_LOUIS','ZIGUINCHOR','KAOLACK','TAMBACOUNDA']
# Capacité installée par zone (MW) et profil solaire
CAPA_SOLAIRE = {'DAKAR_NORD':80,'DAKAR_SUD':60,'THIES':120,'SAINT_LOUIS':150,
                'ZIGUINCHOR':40,'KAOLACK':90,'TAMBACOUNDA':110}
CONSO_BASE   = {'DAKAR_NORD':320,'DAKAR_SUD':280,'THIES':95,'SAINT_LOUIS':75,
                'ZIGUINCHOR':35,'KAOLACK':80,'TAMBACOUNDA':60}

producer = KafkaProducer(bootstrap_servers=['kafka:9092'],
```

```
    value_serializer=lambda v: json.dumps(v).encode())

def gen_conso(zone, heure):
    base  = CONSO_BASE[zone]
    # Pic matin 7-9h et soir 18-22h
    facteur = 1.0 + 0.4*np.sin(np.pi*(heure-7)/12) if 7<=heure<=22 else 0.7
    conso   = round(base * facteur * random.uniform(0.9,1.1), 2)
    return {'compteur_id': f'CPT_{zone}_{uuid.uuid4().hex[:8].upper()}',
            'zone':zone, 'consommation_mw':conso,
            'heure':heure, 'timestamp':datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ')}

def gen_solaire(zone, heure):
    m = datetime.utcnow().month
    # Production solaire : pic 10h-15h, nuit=0
    prod = max(0, CAPA_SOLAIRE[zone] * np.sin(np.pi*(heure-6)/12)
               * (0.8 + 0.2*np.sin(np.pi*m/6))  # saisonnalité
               * random.uniform(0.7,1.0))         # ensoleillement aléatoire
    return {'zone':zone, 'production_mw':round(prod,2),
            'capacite_installee_mw':CAPA_SOLAIRE[zone],
            'heure':heure,'timestamp':datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ')}

if __name__ == '__main__':
    print('Simulateur Watt-SN démarré...')
    while True:
        h = datetime.utcnow().hour
        for zone in ZONES:
            producer.send('watt_conso_raw',  gen_conso(zone, h))
            producer.send('watt_solaire_raw', gen_solaire(zone, h))
        producer.flush()
        time.sleep(5)
```

3.3 Pipeline Spark Streaming — streaming

> ⚠ **Privacy critique**
> Drop des champs PII après anonymisation SHA-256. Les données sensibles ne doivent jamais apparaître dans HBase ou Hive.

streaming_watt_sn.py

```
# streaming_watt.py — Pipeline Spark Streaming Watt-SN
# Prédiction délestage + classification ROUGE/ORANGE/VERT
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sha2, concat, lit, from_json, to_json, struct,
    window, avg, sum as spark_sum, when, coalesce, current_timestamp
)
from pyspark.sql.types import StructType, StructField, StringType, FloatType, IntegerType
from pyspark.ml import PipelineModel

SALT    = os.environ.get('WATT_SECRET_SALT','default_salt')
BROKERS = 'kafka:9092'
spark   = SparkSession.builder.appName('Watt_SN_Streaming') \
    .config('spark.sql.shuffle.partitions','4').getOrCreate()
spark.sparkContext.setLogLevel('WARN')

conso_schema = StructType([
    StructField('compteur_id',     StringType(), True),
    StructField('zone',            StringType(), True),
    StructField('consommation_mw', FloatType(),  True),
    StructField('heure',           IntegerType(),True),
    StructField('timestamp',       StringType(), True),
])
solaire_schema = StructType([
    StructField('zone',                StringType(), True),
    StructField('production_mw',       FloatType(),  True),
    StructField('capacite_installee_mw',FloatType(), True),
    StructField('heure',               IntegerType(),True),
    StructField('timestamp',           StringType(), True),
])

# Lecture Kafka — topic consommation
conso_df = (spark.readStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('subscribe','watt_conso_raw').load()
    .select(from_json(col('value').cast('string'), conso_schema).alias('d')).select('d.*')
    .withColumn('event_ts', current_timestamp())
    # Privacy : anonymisation compteur industriel
    .withColumn('compteur_secure', sha2(concat(col('compteur_id'),lit(SALT)),256))
    .drop('compteur_id')
)

# Lecture Kafka — topic solaire
solaire_df = (spark.readStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('subscribe','watt_solaire_raw').load()
    .select(from_json(col('value').cast('string'),
solaire_schema).alias('d')).select('d.*')
    .withColumn('event_ts', current_timestamp())
)

# Agrégation consommation par zone (fenêtre 1h, watermark 10min)
conso_agg = (conso_df
    .withWatermark('event_ts','10 minutes')
    .groupBy(window('event_ts','1 hour','15 minutes'), 'zone')
    .agg(spark_sum('consommation_mw').alias('conso_totale_mw'),
         avg('consommation_mw').alias('conso_moy_mw'))
)

# Agrégation production solaire (même fenêtre)
solaire_agg = (solaire_df
```

```
    .withWatermark('event_ts','10 minutes')
    .groupBy(window('event_ts','1 hour','15 minutes'), 'zone')
    .agg(spark_sum('production_mw').alias('prod_solaire_mw'))
)

# Classification risque délestage
# (join stream-stream remplacé par 2 queries indépendantes — correction v2)
risque_df = conso_agg.withColumn('risque_delestage',
    when(coalesce(col('conso_totale_mw'),lit(0)) > 900, lit('ROUGE'))
    .when(coalesce(col('conso_totale_mw'),lit(0)) > 700, lit('ORANGE'))
    .otherwise(lit('VERT'))
)

# Écriture alertes → Kafka
q1 = (risque_df.select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('topic','watt_alerts')
    .option('checkpointLocation','/tmp/watt_conso_ckpt')
    .outputMode('update').start())

q2 = (solaire_agg.select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('topic','watt_solaire_agg')
    .option('checkpointLocation','/tmp/watt_solaire_ckpt')
    .outputMode('update').start())

q1.awaitTermination()
```

3.4 Tables Hive & Vues Analytics — hive_setup.sql

Les vues Hive utilisent systématiquement COALESCE sur les colonnes nullables pour éviter les NULL silencieux dans les agrégations.

hive_setup.sql

```
-- hive_setup.sql — Tables Hive Watt-SN
CREATE DATABASE IF NOT EXISTS watt_sn COMMENT 'Énergie Sénégal — UADB 2025';
USE watt_sn;

CREATE TABLE IF NOT EXISTS conso_historique (
    zone STRING, heure INT,
    conso_totale_mw DOUBLE, conso_moy_mw DOUBLE,
    prod_solaire_mw DOUBLE, risque_delestage STRING,
    ingestion_ts TIMESTAMP
) PARTITIONED BY (date_obs STRING)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

-- Vue : Risque délestage par zone (24h glissantes)
CREATE OR REPLACE VIEW vue_risque_zone AS
SELECT zone,
    AVG(COALESCE(conso_totale_mw,0))  AS conso_moy_24h,
    AVG(COALESCE(prod_solaire_mw,0))  AS prod_sol_moy,
    MAX(CASE WHEN risque_delestage='ROUGE' THEN 1 ELSE 0 END) AS a_eu_rouge,
    COUNT(CASE WHEN risque_delestage='ROUGE' THEN 1 END) AS nb_episodes_rouge
FROM conso_historique
WHERE date_obs >= DATE_SUB(CURRENT_DATE(),1)
GROUP BY zone ORDER BY nb_episodes_rouge DESC;

-- Vue : Potentiel solaire non exploité
CREATE OR REPLACE VIEW vue_solaire_optimisation AS
SELECT zone,
    AVG(conso_totale_mw)  AS conso_moy,
    AVG(prod_solaire_mw)  AS prod_sol_moy,
    AVG(conso_totale_mw - COALESCE(prod_solaire_mw,0)) AS deficit_mw
FROM conso_historique
GROUP BY zone
ORDER BY deficit_mw DESC;
```

3.5 DAG Airflow MLOps

Le BranchPythonOperator détecte automatiquement les dérives et déclenche le réentraînement du modèle si nécessaire. Les recommandations sont écrites dans HBase pour un accès temps réel.

dags/watt_sn_dag.py

```
# dags/watt_sn_dag.py — DAG Airflow Watt-SN
# Recalcul prévision délestage toutes les heures + alerte si ROUGE
from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.dummy  import DummyOperator
from airflow.utils.dates      import days_ago
from datetime import timedelta
import subprocess, logging

logger = logging.getLogger('watt_dag')
default_args = {'owner':'seye_ahmed','retries':2,'retry_delay':timedelta(minutes=5)}

def check_rouge_zones(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore', port=10000)
    cur  = conn.cursor()
    cur.execute('SELECT zone, nb_episodes_rouge FROM watt_sn.vue_risque_zone WHERE
nb_episodes_rouge > 3')
```

```
    zones_rouge = cur.fetchall()
    ctx['ti'].xcom_push(key='zones_rouge', value=[z[0] for z in zones_rouge])
    logger.info(f'Zones ROUGE détectées : {zones_rouge}')
    return 'trigger_retrain' if len(zones_rouge) > 2 else 'generate_forecast'

def generate_forecast(**ctx):
    subprocess.run(['spark-submit','--master','spark://spark-master:7077',
        '/opt/models/train_delestage_model.py','--mode','infer'], check=True)

def trigger_retrain(**ctx):
    subprocess.run(['spark-submit','--master','spark://spark-master:7077',
        '/opt/models/train_delestage_model.py','--mode','train'], check=True,
timeout=3600)

with DAG('watt_sn_monitoring', default_args=default_args,
         schedule_interval='0 * * * *', start_date=days_ago(1), catchup=False,
         tags=['watt-sn','energie','delestage']) as dag:
    start  = DummyOperator(task_id='start')
    skip   = DummyOperator(task_id='skip_retrain')
    end    = DummyOperator(task_id='end')
    branch = BranchPythonOperator(task_id='check_rouge_zones',
                                  python_callable=check_rouge_zones,
provide_context=True)
    t_train= PythonOperator(task_id='trigger_retrain',
                            python_callable=trigger_retrain, provide_context=True)
    t_fore = PythonOperator(task_id='generate_forecast',
                            python_callable=generate_forecast, provide_context=True)
    start >> branch >> [t_train, t_fore]
    t_train >> t_fore >> end; t_fore >> end
```

4. Livrables Attendus

#	Livrable	Contenu attendu
1	**Pipeline de Données**	Code source complet (GitHub) + template NiFi. Données visibles dans HDFS, topic Kafka watt_conso_raw actif.
2	**Modèle ML & MLOps**	Script d'entraînement + RMSE/R^2 documentés + DAG Airflow avec BranchPythonOperator fonctionnel.
3	**Catalogue Hive**	Tables créées, vues fonctionnelles avec au moins 1 alerte visible dans HBase en démonstration.
4	**Dashboard & Recommandations**	Dashboard graphique (Matplotlib) + recommandations concrètes contextualisées au Énergie & Transition Solaire sénégalais.

5. Barème de Notation — /20 points

ℹ️ **Barème différencié 4 niveaux**
Chaque critère est évalué selon 4 niveaux de maîtrise. La démonstration live en soutenance est obligatoire. Bonus documentation +2 pts.

Critère	Pts	Insuffisant (0)	Passable (50%)	Bien (75%)	Excellent (100%)
Ingestion NiFi Multi-Sources	3 pts	NiFi absent	1 source/3	2 sources, topics créés	3 sources (Senelec+météo+GIS solaire), 2 topics Kafka distincts
Validation & Privacy	4 pts	Aucune validation	Schéma partiel	Schéma + SHA-256 compteurs industriels	Pandera + anonymisation + 0 ID contrat brut dans HBase
Streaming & Prédiction Délestage	6 pts	Pas de Spark	Lecture seule Kafka	Fenêtres 1h + classification ROUGE/ORANGE/VERT	RandomForest + watermarks cohérents + COALESCE + RMSE documenté
Optimisation Solaire & Analytics	4 pts	Pas de Hive	Tables vides	Vues Hive opérationnelles	Dashboard délestage + injection solaire optimisée par zone
Alertes & Recommandations	3 pts	Aucune alerte	Alertes génériques	Alertes par zone géographique	Recommandations délestage tournant équitable par zone + HBase live
TOTAL	**20 pts**				

Bonus Documentation (jusqu'à +2 pts)

Élément	Points bonus
README.md avec docker compose up + captures d'écran	+0.5 pt
Commentaires de code en français + docstrings	+0.5 pt
Tests pytest sur les schémas Pandera et les UDFs	+0.5 pt
Démo live d'une alerte HBase sans erreur en soutenance	+0.5 pt

⚡ **Bon courage pour Watt-SN !**

"Always be a solution, never a problem."

Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI Engineer | Groupe Sonatel